

CONSUMER DATA PLATFORM

TABLE OF CONTENTS

- CONSUMER DATA PLATFORM.....1
 - TABLE OF CONTENTS..... 1
 - 1. EXECUTIVE SUMMARY..... 3
 - Core Purpose.....3
 - Target Outcomes.....3
 - 2. SYSTEM OVERVIEW.....3
 - 2.1 What is a Consumer Data Platform?..... 3
 - 2.2 Why This Architecture?.....4
 - 3. BUSINESS OBJECTIVES & PROBLEM STATEMENT..... 4
 - 3.1 Current Challenges.....4
 - 3.2 Solution Requirements.....5
 - 4. SOLUTION ARCHITECTURE..... 5
 - 4.1 Architecture Principles..... 5
 - 4.2 Technology Decisions.....6
 - 5. HIGH-LEVEL SYSTEM FLOW..... 7
 - 5.1 Data Flow Diagram..... 7
 - 5.2 Data Transformation Journey..... 9
 - 6. DETAILED COMPONENT DESCRIPTIONS..... 10
 - 6.1 MySQL - Transactional Source System.....10
 - 6.2 Kafka - Real-Time Event Stream.....11
 - 6.3 Cassandra - Operational Analytics Store.....12
 - 6.4 Parquet Data Lake - Analytical Storage.....13
 - 7. DATA MODEL & SCHEMA DESIGN..... 14
 - 7.1 Entity-Relationship Diagram..... 14
 - 7.2 Schema Specifications.....14
 - 8 Microservices Overview..... 17
 - 8.1 MySQL Data Generation Service.....17
 - 8.2 EVENT GENERATOR (Akka)..... 18
 - 8.3 PIPELINE ARCHITECTURE & SPECIFICATIONS..... 20
 - 8.3.1 Overview of Three Pipelines..... 20
 - 8.3.2 Pipeline A: Customer Profile Consolidation..... 20
 - 8.3.3 Pipeline B: Daily Transaction Summary.....25
 - 8.3.4 Pipeline C: Kafka Event Archival (Streaming).....27
 - 9. DATA FLOW & EXECUTION TIMELINE..... 31
 - 9.1 Complete Daily Execution Flow..... 31

9.2 Detailed Process Flow with Dependencies.....	32
9.3 Critical Timeline Constraints.....	33
10. API LAYER SPECIFICATIONS.....	33
10.1 Overview of REST APIs.....	33
10.2 API Endpoint 1: Get Customer Profile.....	34
10.3 API Endpoint 2: Get Daily Transaction Summary.....	36
10.4 API Endpoint 3: Get Customer Events.....	40
10.5 API Performance Characteristics.....	45
11. DATA QUALITY & GOVERNANCE FRAMEWORK.....	46
11.1 Data Quality Levels.....	46
11.2 Quality Metrics & KPIs.....	46
11.3 Data Governance Policies.....	47
13. SUCCESS CRITERIA & KEY METRICS.....	48
13.1 Functional Success Criteria.....	48
13.2 Performance Targets.....	48
13.3 Operational Metrics.....	49
14. PROJECT DELIVERABLES.....	50
14.1 Code & Implementation Deliverables.....	50
14.2 Documentation Deliverables.....	51
14.3 Test & Validation Deliverables.....	51
14.4 Data Deliverables.....	51
15. LEARNING OUTCOMES.....	52
15.1 Data Engineering Concepts.....	52
15.2 Technology Platforms Mastered.....	52
15.3 Software Engineering Practices.....	53

1. EXECUTIVE SUMMARY

The Consumer Data Platform is a unified, multi-layered data engineering ecosystem designed to consolidate customer master data, transactional records, and behavioral events from disparate sources into actionable analytical datasets. The platform processes data through multiple specialized pipelines that deliver real-time customer profiles, daily transaction summaries, and comprehensive event archives.

Core Purpose

Enable business teams to quickly answer critical questions:

- "What is customer XYZ's lifetime value and purchase pattern?"
- "What products performed best yesterday?"
- "How are customers interacting with our platform in real-time?"
- "Which customer segments are most valuable?"

Target Outcomes

- Consolidated view of 5,000+ customers across all touchpoints
- Historical transaction analysis spanning 4+ years
- Real-time behavioral event tracking at 10 events/second
- Sub-100ms customer profile lookups
- Fully archived event history for replay and compliance

2. SYSTEM OVERVIEW

2.1 What is a Consumer Data Platform?

A Consumer Data Platform (CDP) is an integrated technology stack that:

Ingests data from multiple sources (transactional systems, event streams, user interactions)

Processes raw data through transformation pipelines (joins, aggregations, enrichment)

Stores processed data in optimized formats for different use cases (operational vs. analytical)

Serves curated datasets to downstream applications (dashboards, APIs, machine learning models)

2.2 Why This Architecture?

Traditional approaches struggle with:

- Data scattered across systems (customer data in MySQL, events in Kafka, nothing connected)
- Slow queries taking hours to answer simple questions
- Manual data processing and report generation
- No real-time visibility into customer behavior
- Difficulty replaying historical events

This platform solves these by:

- Consolidating all data into a unified ecosystem
- Pre-computing metrics for instant access
- Automating daily data pipelines
- Providing real-time streaming capabilities
- Enabling historical analysis with full event replay

3. BUSINESS OBJECTIVES & PROBLEM STATEMENT

3.1 Current Challenges

Data Fragmentation

- Customer master data lives in MySQL
- Transaction history scattered across files and databases
- Behavioral events stream through Kafka with no permanent storage
- No single source of truth for customer information

Analytical Bottlenecks

- Customer lifetime value queries take 30+ minutes
- Daily reports require manual joins and aggregations
- No real-time customer profile updates
- Historical event data is lost after 7 days

Operational Inefficiencies

- Business users wait hours for analytical answers
- Data quality issues go undetected
- No audit trail for behavioral events
- Duplicate data processing and storage

3.2 Solution Requirements

Requirement 1: Unified Customer View

- Consolidate customer master data with transaction history
- Pre-compute key metrics (lifetime spend, purchase patterns, preferences)
- Serve profiles in <100 milliseconds

Requirement 2: Daily Analytical Summaries

- Generate daily transaction summaries by customer
- Enable trend analysis and performance tracking
- Partition data by date for efficient querying

Requirement 3: Event Archival & Replay

- Capture all behavioral events in real-time
- Archive to data lake for long-term storage
- Enable event replay for compliance and analysis

Requirement 4: API Access

- Expose customer profiles via REST API
- Provide transaction summary endpoints
- Enable event retrieval for dashboards

Requirement 5: Scalability

- Handle current load: 5K customers, 500K-1M transactions
- Scale to 10x volume without architectural changes
- Support sustained 10 events/second throughput

4. SOLUTION ARCHITECTURE

4.1 Architecture Principles

Separation of Concerns

- Each pipeline has a single, well-defined responsibility
- Different storage systems optimized for different access patterns
- Clear interfaces between components

Scalability

- Distributed processing for large datasets
- Partitioned storage for efficient querying

Reliability

- Data quality checks at every stage
- Multiple replicas for high availability
- Automated recovery and retry mechanisms

Auditability

- Complete event history preserved
- Data lineage tracked through all pipelines
- Quality metrics and validation logs

4.2 Technology Decisions

Why MySQL for Source Data?

- ACID compliance ensures data consistency
- Normalized schema reduces storage
- Mature ecosystem with excellent tools
- Fast writes for transactional data

Why Cassandra for Customer Profiles?

- Optimized for fast reads at scale
- Supports denormalized data models
- Built for high availability (replication factor 3)
- Sub-100ms response time guarantees

Why Parquet for Data Lake?

- Columnar format enables efficient analytical queries
- Snappy compression reduces storage by 80%
- Partitioning supports date-range queries
- Standard format compatible with all tools

Why Kafka for Events?

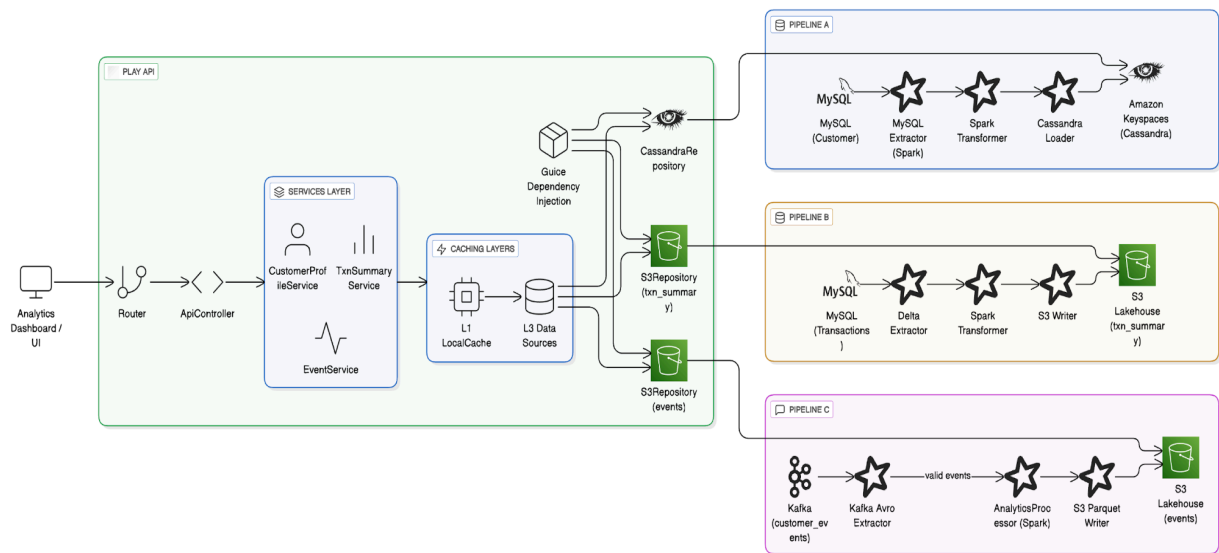
- Real-time event streaming capabilities
- Event replay through retention period
- Decouples producers from consumers
- Proven at scale (10+ events/second)

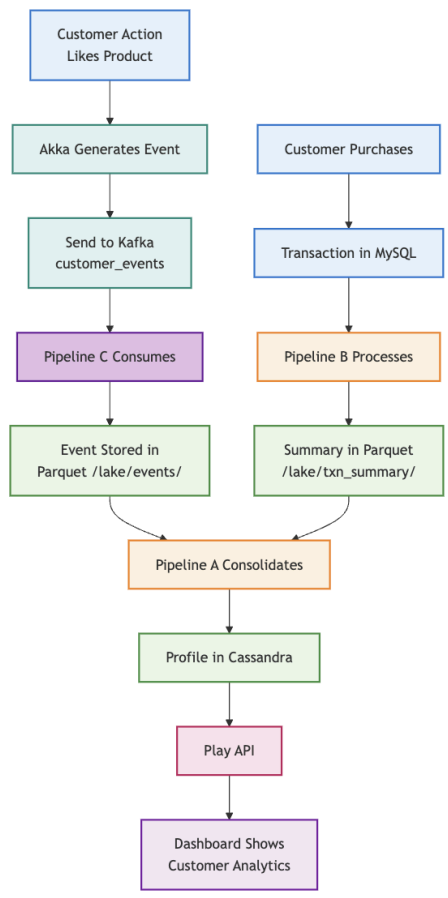
Why Spark for Processing?

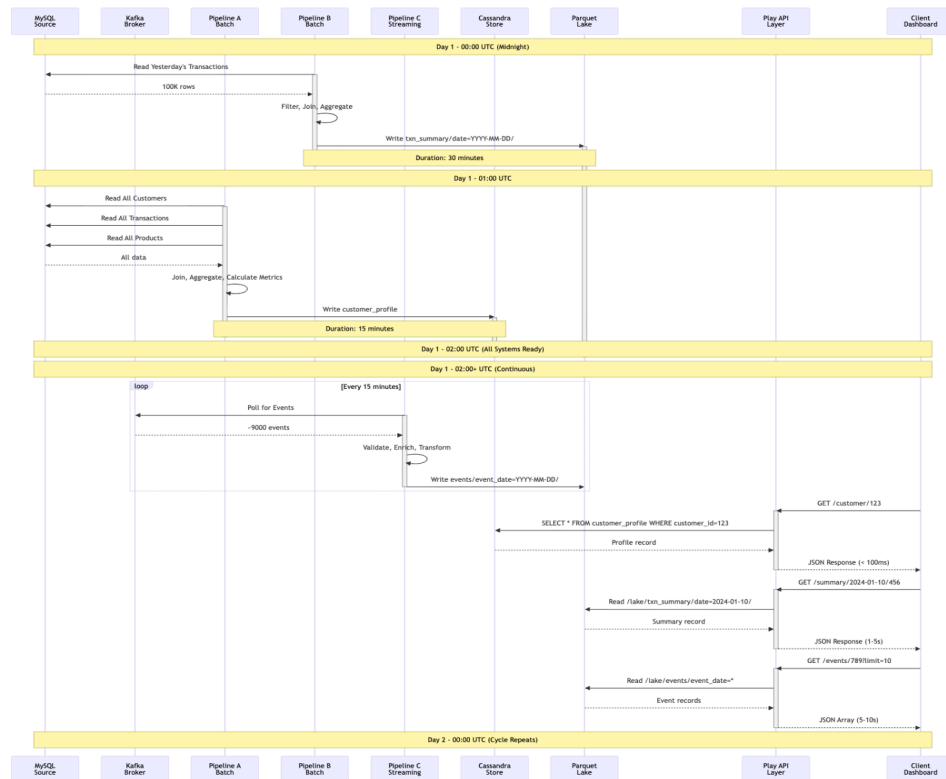
- Unified batch and streaming engine
- Handles large datasets efficiently
- Rich transformation libraries
- Active community and ecosystem

5. HIGH-LEVEL SYSTEM FLOW

5.1 Data Flow Diagram







5.2 Data Transformation Journey

Stage 1: Ingestion

- Raw data enters from MySQL, Kafka, and external sources
- Stored in original format without transformation
- No data loss or filtering

Stage 2: Validation

- Schema validation ensures data conforms to expected structure
- Quality checks identify anomalies (negative amounts, missing references)
- Invalid records logged for investigation
- Clean data passes to transformation

Stage 3: Transformation

- Multiple data sources joined together
- Aggregations computed (sums, counts, modes)
- Denormalized datasets created for fast access
- Metrics calculated for different analytical needs

Stage 4: Storage

- Processed data stored in optimized formats
- Cassandra for real-time lookups
- Parquet for historical analysis
- Partitioned for efficient querying

Stage 5: Serving

- APIs expose curated datasets
- Dashboards consume pre-computed metrics
- Sub-second response times for operational needs
- Seconds-range latency for analytical queries

6. DETAILED COMPONENT DESCRIPTIONS

6.1 MySQL - Transactional Source System

Purpose: Store and manage operational business data

Data Stored:

- Customer master records (names, emails, signup dates)
- Product catalog with pricing and categorization
- Complete transaction history (2020-2024)

Key Characteristics:

- Normalized schema to minimize storage
- ACID compliance ensures consistency
- Indexes on frequently queried columns
- Primary and foreign key constraints enforce integrity

Access Pattern:

- Daily read of entire dataset for Pipeline A
- Daily read of yesterday's transactions for Pipeline B
- Baseline data source for all analyses

Scale:

- 5,000 customer records
- 500 product records
- 200k transaction records
- Estimated storage: 200-500 MB

6.2 Kafka - Real-Time Event Stream

Purpose: Capture and stream user behavioral events

Events Captured:

- LIKE: User marks product as favorite
- WISHLIST: User adds product to wish list
- CART_ADD: User adds product to shopping cart

Event Properties:

- Unique event ID (UUID)
- Customer ID (1-5000 range)
- Product ID (1-500 range, optional for some events)
- Event timestamp (ISO8601 UTC)
- Generated every 100 milliseconds (10 events/second)

Key Characteristics:

- 10 partitions for parallel consumption
- 7-day retention for replay capability
- Snappy compression for storage efficiency
- Real-time throughput: 864K events per day

Access Pattern:

- Continuous consumption by Pipeline C
- Append-only log structure enables replay
- No direct query interface

Benefits:

- Decouples event producers from processors
- Enables multiple consumers independently
- Provides event replay capability for recovery
- Audit trail for all user interactions

6.3 Cassandra - Operational Analytics Store

Purpose: Serve pre-computed customer profiles for instant lookup

Data Model:

- Single table: customer_profile
- Partition key: customer_id

Stored Metrics:

- total_spend: Lifetime customer spending
- total_transactions: Number of purchases
- avg_order_value: Average transaction amount
- first_purchase: Date of first transaction
- last_purchase: Date of most recent transaction
- favorite_category: Most frequently purchased category

Key Characteristics:

- Optimized for read operations
- <100 millisecond response time
- Replication factor 3 for high availability
- No complex joins needed (denormalized)
- Updated daily via Pipeline A

Access Pattern:

- Fast lookups by customer_id
- Used by API /customer/{id} endpoint
- Cache-friendly due to small result sets
- Single-row operations only

Scale:

- 5,000 customer records
- Storage per row: ~500 bytes
- Total size: ~2.5 MB
- Query throughput: 1000+ per second

6.4 Parquet Data Lake - Analytical Storage

Purpose: Archive and serve data for analytical queries

Datasets Stored:

Dataset 1: Daily Transaction Summaries

- Path: /lake/txn_summary/date=YYYY-MM-DD/
- Partitioned by date for efficient filtering
- One row per (customer, date) combination
- Metrics: total_amount, total_items, distinct_products, top_category

Dataset 2: Raw Event Archive

- Path: /lake/events/event_date=YYYY-MM-DD/
- Partitioned by event_date
- Complete event details with ingestion timestamp
- Enables historical event analysis and replay

Key Characteristics:

- Columnar format optimized for analytical queries
- Snappy compression reduces storage by 80%
- Partition pruning enables date-range queries
- Immutable append-only writes

Access Pattern:

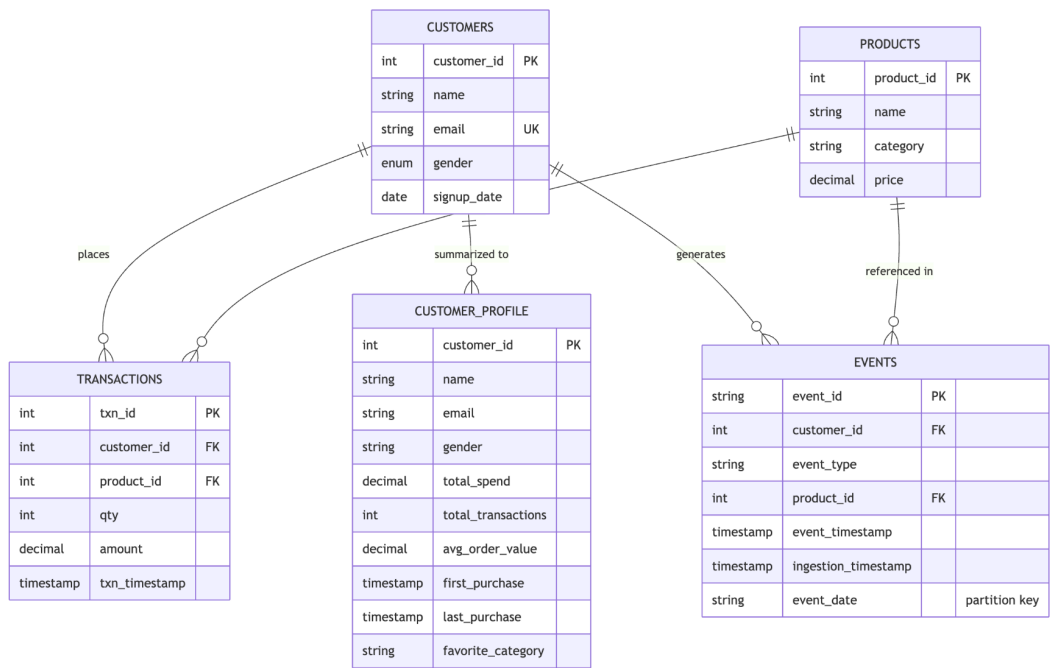
- Batch analytical queries over date ranges
- Time-series analysis and trend detection
- Exploratory data analysis
- Historical reporting

Scale:

- Daily transaction summary: 5K rows
- Daily events: 864K rows
- Monthly storage: 50-100 GB (compressed)
- Query latency: 1-10 seconds with partition pruning

7. DATA MODEL & SCHEMA DESIGN

7.1 Entity-Relationship Diagram



7.2 Schema Specifications

CUSTOMERS Table

Field	Type	Constraints	Purpose
customer_id	INT	PRIMARY KEY	Unique customer identifier
name	VARCHAR(100)	NOT NULL	Customer full name
email	VARCHAR(150)	UNIQUE, NOT NULL	Email address, unique across system
gender	ENUM('M','F','O')	NOT NULL	Gender classification
signup_date	DATE	NOT NULL	Account creation date

PRODUCTS Table

Field	Type	Constraints	Purpose
product_id	INT	PRIMARY KEY	Unique product identifier
name	VARCHAR(100)	NOT NULL	Product name
category	VARCHAR(50)	NOT NULL	Product category (10 fixed values)
price	DECIMAL(10,2)	NOT NULL	Unit price in dollars

TRANSACTIONS Table

Field	Type	Constraints	Purpose
txn_id	INT	PRIMARY KEY	Unique transaction identifier
customer_id	INT	FOREIGN KEY	Reference to customers table
product_id	INT	FOREIGN KEY	Reference to products table
qty	INT	NOT NULL, >0	Quantity purchased (1-10)
amount	DECIMAL(10,2)	NOT NULL, >0	Total transaction amount
txn_timestamp	TIMESTAMP	NOT NULL	Purchase timestamp

CUSTOMER_PROFILE Table (Cassandra)

Column	Type	Key Type	Purpose
customer_id	INT	Partition Key	Unique customer identifier
name	TEXT		Customer name
email	TEXT		Email address
gender	TEXT		Gender
total_spend	DECIMAL		Sum of all transaction amounts

total_transactions	INT		Count of transactions
avg_order_value	DECIMAL		total_spend / total_transactions
first_purchase	TIMESTAMP		Earliest transaction date
last_purchase	TIMESTAMP		Most recent transaction date
favorite_category	TEXT		Mode of product categories

TRANSACTION_SUMMARY (Parquet)

Column	Type	Partition	Purpose
date	DATE	YES	Transaction date
customer_id	INT	NO	Customer identifier
total_amount	DECIMAL	NO	Daily spending total
total_items	INT	NO	Daily quantity total
distinct_products	INT	NO	Number of unique products
top_category	STRING	NO	Most purchased category

EVENTS Table (Parquet)

Column	Type	Partition	Purpose
event_id	STRING	NO	UUID of event
customer_id	INT	NO	Customer who performed event
event_type	STRING	NO	LIKE, WISHLIST, or CART_ADD
product_id	INT	NO	Product reference (nullable)
event_timestamp	TIMESTAMP	NO	When event occurred
ingestion_timestamp	TIMESTAMP	NO	When event was processed
event_date	STRING	YES	Partition: YYYY-MM-DD

8 Microservices Overview

8.1 MySQL Data Generation Service

Purpose: Generate and load synthetic datasets for customer analytics platform testing and development.

Technology Stack: Apache Spark 3.5, Scala 2.13, MySQL 8.0, JDBC Connector

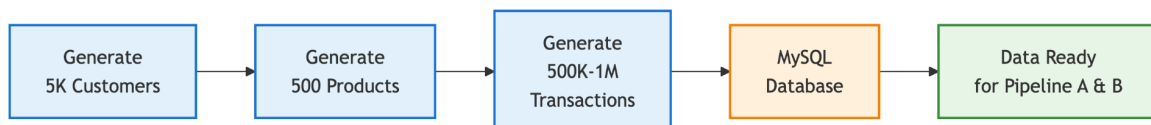
Architecture Pattern: ETL pipeline with in-memory generation, batch validation, and optimized JDBC writes.

Key Features:

- Parallel data generation using Spark distributed computing
- Multi-stage validation ensuring data quality before load
- Optimized MySQL writes with batch processing and connection pooling
- Performance monitoring with detailed metrics tracking
- Idempotent operations supporting safe re-execution

Performance Characteristics:

- Generates 505,500 records (5K customers, 500 products, 500K transactions)
- Completes end-to-end in 40-55 seconds on standard hardware
- Achieves sustained throughput of 10,000-15,000 rows/second
- Maintains 100% data quality validation pass rate
- Scales linearly to 1M+ transactions



```

=====
PERFORMANCE SUMMARY
=====
Customer Generation      Records    5000 Duration    0.01s Throughput    500000 rows/s
Product Generation      Records     500 Duration    0.01s Throughput    38461 rows/s
Transaction Generation   Records  200000 Duration    0.07s Throughput   2702702 rows/s
Customer Load MySQL      Records    5000 Duration   13.79s Throughput     362 rows/s
Product Load MySQL       Records     500 Duration   10.38s Throughput      48 rows/s
Transaction Load MySQL   Records  200000 Duration   30.42s Throughput    6573 rows/s
=====
TOTAL EXECUTION TIME 76.94 seconds
TOTAL RECORDS GENERATED 205500
TOTAL RECORDS LOADED 205500
=====

Database Verification
Customers in DB          5000 Expected 5000
Products in DB           500 Expected 500
Transactions in DB      200000 Expected 200000

SUCCESS All data loaded and verified correctly
=====
13:07:16.178 [sbt-bg-threads-1] INFO Application$ -- APPLICATION COMPLETED SUCCESSFULLY
13:07:16.184 [shutdown-hook-0] INFO org.apache.spark.SparkContext -- Invoking stop() from shutdown hook

```

8.2 EVENT GENERATOR (Akka)

Purpose

Simulate 10 events/second to Kafka topic

Implementation

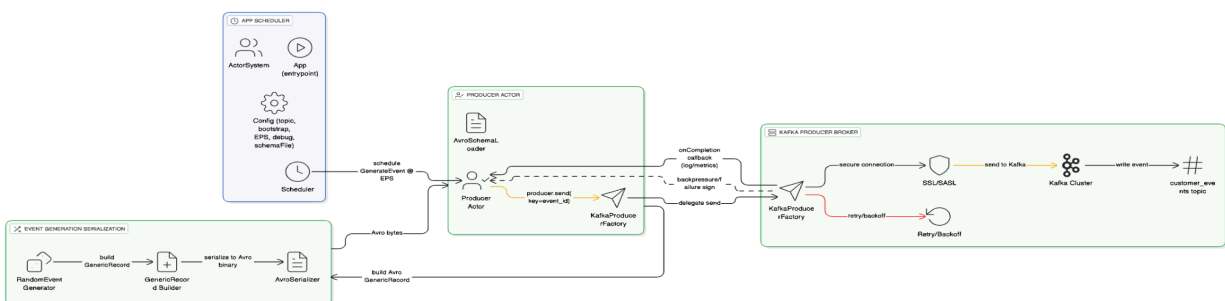
- Akka actors generate random events
- Customer IDs match MySQL customers (1-5000)
- Product IDs match MySQL products (1-500)
- Event types: LIKE, WISHLIST, CART_ADD (random)
- Timestamp: current time
- Sends to Kafka topic: customer_events

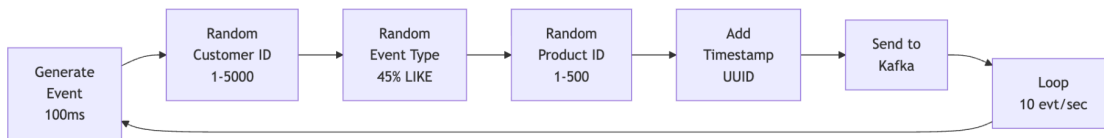
Example Generated Events

Event 1: customer_id=123, event_type=LIKE, product_id=456
 Event 2: customer_id=789, event_type=CART_ADD, product_id=321
 Event 3: customer_id=456, event_type=WISHLIST, product_id=null
 Event 4: customer_id=234, event_type=LIKE, product_id=789

...

(10 events per second = 864,000 per day)





```
.616 [kafka-producer-network-thread | producer-1] INFO o.apache.kafka.clients.NetworkClient - [Producer clientId=producer-1] Node -3 disconnected.
.617 [kafka-producer-network-thread | producer-1] WARN o.apache.kafka.clients.NetworkClient - [Producer clientId=producer-1] Connection to node -3 (localhost:9092) cannot
not be established. Node may not be available.
.618 [kafka-producer-network-thread | producer-1] WARN o.apache.kafka.clients.NetworkClient - [Producer clientId=producer-1] Bootstrap broker localhost:9092 is not
connected
.730 [kafka-producer-network-thread | producer-1] INFO org.apache.kafka.clients.Metadata - [Producer clientId=producer-1] Cluster ID: utc3eCjXtsah_z6Clk2_uQ
.520 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 100
.517 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 200
.518 [event-generator-system-akka.actor.default-dispatcher-5] INFO a.BehaviouralEventGeneratorActor - Total events generated: 300
.528 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 400
.517 [event-generator-system-akka.actor.default-dispatcher-5] INFO a.BehaviouralEventGeneratorActor - Total events generated: 500
.517 [event-generator-system-akka.actor.default-dispatcher-5] INFO a.BehaviouralEventGeneratorActor - Total events generated: 600
.516 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 700
.519 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 800
.525 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 900
.515 [event-generator-system-akka.actor.default-dispatcher-5] INFO a.BehaviouralEventGeneratorActor - Total events generated: 1000
.514 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 1100
.513 [event-generator-system-akka.actor.default-dispatcher-4] INFO a.BehaviouralEventGeneratorActor - Total events generated: 1200
```

```
HC879713-8626-4387-8567-912208712100-?LIKE?????fffff
H81d6f3c7-c709-4165-bb80-e7c20566b1f7?LIKE?????fffff
Hf1919ec2-5beb-46d4-b70b-c52f02409c1c?*LIKE?????fffff
Hc2beade2-25f8-4685-9e70-95e24f5a31f8?CART_ADD?????ff
H0b7a2c75-ac0a-4493-a514-50792d35beb7?WISHLIST?????ff
H1ae98c09-0db3-4f13-8f4a-5bd6f5c96001?LIKE?????fffff
H453209b4-cd96-4581-bdce-3809d3d61906?PwISHLIST?????ff
H73a51872-e253-4263-915c-0c149159bddf?WISHLIST?????ff
He84df5b8-1aed-45d6-9894-cc1732206d62?LIKE?????ff?c?fffff
H865bb080-abad-4954-996e-3f042b33e071?WISHLIST?????ff
LIKE?????f8d4a-49cc-a47a-8c0c8fef55d5?NLIKE?????ff?fffff
H28a7a07f-fb72-4781-8561-1e68e160e569?WISHLIST?????ff
H5d26b111-a2fe-44e2-900d-5e440c0b2a11?LIKE?????ff?fffff
H9b4da157-dc41-456f-956c-591c09177ebd?/LIKE?????ff?fffff
H8c509197-b629-4925-8f5d-2aba12eddd10?WISHLIST?????ff
H2df40efb-b04a-4712-a743-0ad909998465?#WISHLIST?????ff
Hf891aa06-19cc-4bba-9d7c-d8eabbfa1044?/LIKE?????ff?fffff
Hc4b049d3-d936-4dc3-853b-79e20a3ad4ac?&L>WISHLIST?????ff
H64669f9d-1f93-4bfd-9674-e7c54bbb0259?WISHLIST?????ff
H84327660-1ad5-4b75-b17e-31e41ae7620b?<LIKE?????ff?fffff
Hccb152bd-7d8d-4fee-9236-9609005039df?22?CART_ADD?????ff
H8aa654ae-a161-4b43-a186-84a4a73726ab?F.CART_ADD_?????ff
Hcd2107b-a11f-45ea-a5be-a3e8b0b77c92?!?CART_ADD?????ff
H1a1651d9-7453-4fda-a26a-95fd8761e6322A?WISHLIST?????ff
H8b9a6952-499f-43e4-b6a7-65cbbcedcd87?/LIKE?????ff?fffff
Hf6abbf2d-fca1-4483-9b46-a4919b1ba984?WISHLIST?????ff
H70ba4045-f5fa-4688-8f03-10d4e667a07f?<LIKE?????ff?fffff
He9e9a261-2a1d-4c4f-901a-19c6aeb5518c?3?CART_ADD?S?????ff
H8ff804b7-f732-44ca-bfe8-05396b2478f8?0?CART_ADD_?S?????ff
H4bc6b9b1-e438-4eab-8d49-1a0438d07000?/LIKE?????ff?6?fffff
Heb0deb55-1709-4bba-80a7-cb22ad11ea24?7N?CART_ADD?????ff
H2ada80c0-783a-44ba-8f15-0f8d8258ccce?CART_ADD?????ff
H3d42be20-1620-4d10-98af-2b2c5e620dc9?I?WISHLIST?????ff
Hfb3c6291-7d84-43cd-ac92-295c0e91785e?-/LIKE?????ff
H2a260b34-ff98-48da-8a12-441897b852fa?6/LIKE?????ff
H1ba3081a-b019-4ef0-bed6-339e837268d2?+?CART_ADD?S?????ff
H295038d9-47bd-441b-b3fe-590425ccc70f?L?CART_ADD?????ff
Hdbb0953c-7e1b-4efc-a64e-8ea6504713f5?=?WISHLIST?????ff
Ha45a8c27-1a50-45f8-bfb4-087006534ea4?<LIKE?????ff
H129e882a-97ac-437d-aa81-83515e1566d2?+?/LIKE?????ff
Hb31cfa8b-f7bc-4e5d-9104-d21382a1cc60?/LIKE?????ff
```

8.3 PIPELINE ARCHITECTURE & SPECIFICATIONS

8.3.1 Overview of Three Pipelines

The platform consists of three complementary pipelines, each with distinct purpose, schedule, and characteristics:

Aspect	Pipeline A	Pipeline B	Pipeline C
Name	Customer Profile	Daily Summary	Event Archive
Type	Batch	Batch	Streaming
Trigger	Daily 1 AM UTC	Daily 12 AM UTC	Continuous
Input	All MySQL data	Yesterday transactions	Kafka stream
Output	Cassandra profiles	Parquet summaries	Parquet events
Duration	15 minutes	30 minutes	24/7 continuous
Frequency	Once daily	Once daily	Every 15 min
Data Volume	5K rows → 5K profiles	100K rows → 100K summaries	10 events/sec
Purpose	Real-time lookup	Analytical reporting	Event archival

8.3.2 Pipeline A: Customer Profile Consolidation

Objective: Build comprehensive customer profiles for operational lookups

Input Sources:

- MySQL customers table (5K rows)
- MySQL transactions table (500K-1M rows)
- MySQL products table (500 rows)

Output Target:

- Cassandra customer_profile table (5K rows)

Execution Schedule: Daily at 01:00 UTC | Max Duration: 15 minutes

Transformation Steps:

Step 1: Extract Phase

- Read all records from customers table
- Read complete transaction history
- Read all product master data
- No filtering at this stage

Step 2: Data Quality Validation

- Reject records with null customer_id or product_id
- Remove transactions with negative amounts
- Remove duplicate transactions by txn_id
- Log discarded records with rejection reason

Step 3: Multi-Source Join

- Inner join transactions with customers (enforce customer existence)
- Inner join transactions with products (enforce product existence)
- Result: enriched transaction records with all customer and product attributes

Step 4: Aggregation by Customer

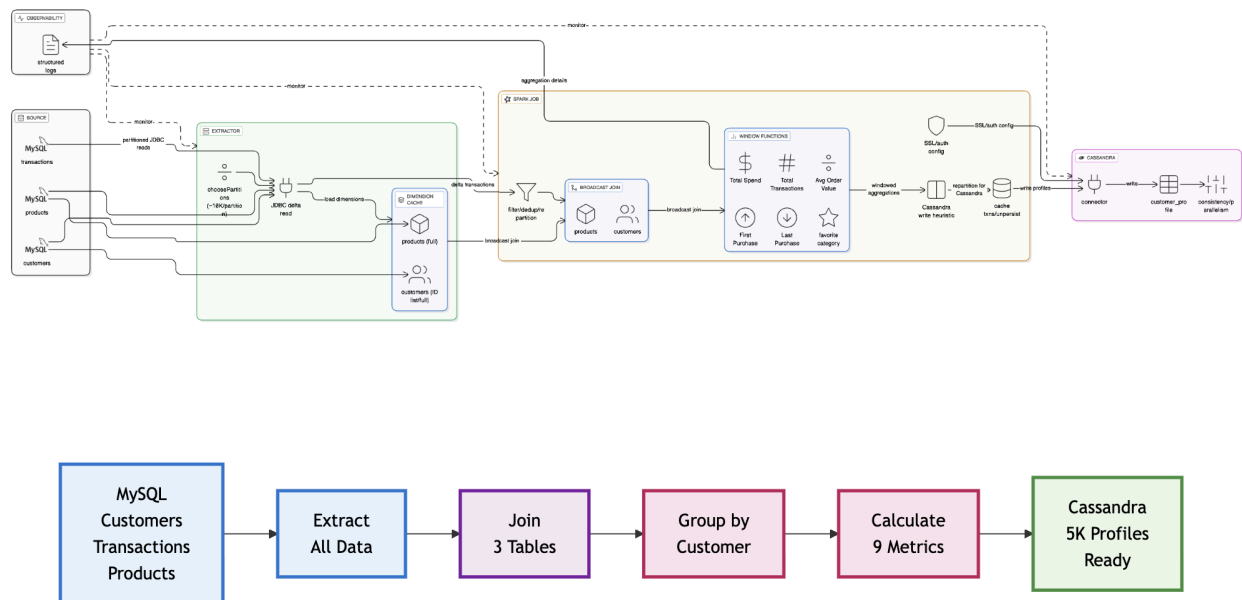
- Group all transactions by customer_id
- Compute metrics for each customer:
 - total_spend: SUM of all transaction amounts
 - total_transactions: COUNT of transactions
 - avg_order_value: total_spend divided by total_transactions
 - first_purchase: MIN transaction timestamp
 - last_purchase: MAX transaction timestamp
 - favorite_category: MODE (most frequent) product category

Step 5: Load Phase

- Write customer profiles to Cassandra
- Overwrite previous day's profiles (complete refresh)
- Verify write acknowledgments
- Log profile count and data quality metrics

Data Quality Checks:

Check	Condition	Action	Impact
Null Primary Key	customer_id IS NULL	Discard record	High
Missing Customer Ref	customer_id NOT IN customers	Discard record	High
Missing Product Ref	product_id NOT IN products	Discard record	High
Negative Amount	amount < 0	Filter + Log	Medium
Duplicate Txn	Same txn_id	Keep first only	Medium
Future Date	txn_timestamp > NOW()	Log warning	Low
Zero Transactions	total_transactions = 0	Log warning	Low



Expected Outputs:

- 5,000 customer profile records in Cassandra
- ~15 minutes execution time
- Data quality report with metric counts
- Profile refresh timestamp for monitoring

```
2025-12-15 12:46:40.696 [sbt-bg-threads-1] INFO Extractor.MarketData - Extracted 161523 transactions in 6.0s
2025-12-15 12:46:40.696 [sbt-bg-threads-1] INFO transformer.DataTransformer - Starting transformation pipeline
2025-12-15 12:46:40.696 [sbt-bg-threads-1] INFO transformer.DataQualityValidator - Validating customers
2025-12-15 12:46:44.388 [sbt-bg-threads-1] INFO transformer.DataQualityValidator - Valid customers count: 5000
2025-12-15 12:46:44.389 [sbt-bg-threads-1] INFO transformer.DataQualityValidator - Validating products
2025-12-15 12:46:46.973 [sbt-bg-threads-1] INFO transformer.DataQualityValidator - Valid products count: 500
2025-12-15 12:46:53.863 [sbt-bg-threads-1] INFO transformer.DataQualityValidator - Validating transactions
2025-12-15 12:46:59.984 [sbt-bg-threads-1] INFO transformer.DataQualityValidator - Total transactions before validation: 161523
2025-12-15 12:46:59.985 [sbt-bg-threads-1] INFO transformer.DataQualityValidator - Transactions valid: 161523, invalid: 0, rate: 100.00%
2025-12-15 12:47:09.262 [sbt-bg-threads-1] INFO transformer.ProfileAggregator - Enriching transactions with product categories and aggregating per customer
2025-12-15 12:47:15.391 [sbt-bg-threads-1] INFO transformer.ProfileAggregator - Enriched transactions count: 161523
2025-12-15 12:47:22.105 [sbt-bg-threads-1] INFO transformer.ProfileAggregator - Computed transaction metrics for customers: 5000
2025-12-15 12:47:25.681 [sbt-bg-threads-1] INFO transformer.ProfileAggregator - Computed favorite categories for customers: 5000
2025-12-15 12:47:25.690 [sbt-bg-threads-1] INFO transformer.ProfileAggregator - Built profiles count: 5000
2025-12-15 12:47:25.691 [sbt-bg-threads-1] INFO transformer.DataTransformer - Transformation completed
2025-12-15 12:47:25.867 [sbt-bg-threads-1] INFO loader.CassandraLoader - Initializing Cassandra keyspace=retail, table=customer_profile
2025-12-15 12:47:25.867 [sbt-bg-threads-1] INFO c.d.o.d.i.c.DefaultMavenCoordinates - DataStax Java driver for Apache Cassandra(R) (com.datastax.oss:java-driver-core-shaded) version 4.13.0
2025-12-15 12:47:26.486 [sbt-admin-0] INFO c.d.o.d.internal.core.time.Clock - Could not access native clock (see debug logs for details), falling back to Java system clock
2025-12-15 12:47:40.821 [sbt-bg-threads-1] INFO c.d.s.c.cql.CassandraConnector - Connected to Cassandra cluster.
2025-12-15 12:47:41.389 [sbt-bg-threads-1] INFO loader.CassandraLoader - Cassandra schema initialization complete
2025-12-15 12:47:48.976 [sbt-bg-threads-1] INFO loader.CassandraLoader - Writing 5000 profiles to Cassandra (keyspace=retail, table=customer_profile)
2025-12-15 12:48:14.584 [sbt-bg-threads-1] INFO loader.CassandraLoader - Profiles written to Cassandra successfully
2025-12-15 12:48:14.586 [sbt-bg-threads-1] INFO ProfileConsolidationJob$ - Pipeline A completed successfully
2025-12-15 12:48:14.586 [sbt-bg-threads-1] INFO util.SparkSessionFactory$ - Stopping Spark session
2025-12-15 12:48:14.594 [sbt-bg-threads-1] INFO o.s.jetty.server.AbstractConnector - Stopped Spark@4ca2cc87{HTTP/1.1, (http/1.1)}{0.0.0.0:4040}
[success] Total time: 118 s (0:01:58.0), completed 15-Dec-2025, 12:48:14 PM
```

customer_id ▾	avg_order_value ▾	email	favorite_category	first_purchase	gender	last_purchase	name	total_spend ▾	total_orders ▾
4457	19068.67	customer_4457@example.com	Books	2020-12-22 08:46:13.0+0000	M	2024-12-27 00:18:56.0+0000	Customer_4457	572060.16	30
4518	33734.42	customer_4518@example.com	Automotive	2020-12-30 23:28:36.0+0000	F	2024-11-08 08:57:54.0+0000	Customer_4518	1281908.12	38
2656	26033.45	customer_2656@example.com	Toys	2021-02-16 20:56:24.0+0000	F	2024-12-09 09:15:09.0+0000	Customer_2656	728936.48	28
3215	34477.22	customer_3215@example.com	Automotive	2021-06-10 13:50:06.0+0000	F	2024-10-24 10:48:19.0+0000	Customer_3215	689544.46	20
1836	22856.68	customer_1836@example.com	Sports	2020-12-16 07:30:08.0+0000	F	2024-12-11 15:22:06.0+0000	Customer_1836	777127.12	34
808	24730.07	customer_808@example.com	Automotive	2020-12-17 07:51:03.0+0000	M	2024-12-23 02:13:22.0+0000	Customer_808	816092.31	33
2513	32447.97	customer_2513@example.com	Health	2021-01-02 22:36:46.0+0000	F	2024-12-28 05:24:55.0+0000	Customer_2513	1200574.94	37
3822	24586.06	customer_3822@example.com	Health	2020-12-21 04:18:35.0+0000	O	2024-10-30 16:14:09.0+0000	Customer_3822	712995.86	29
2836	23073.45	customer_2836@example.com	Electronics	2021-02-06 07:30:30.0+0000	M	2024-12-31 09:23:51.0+0000	Customer_2836	899864.64	39
4347	24949.78	customer_4347@example.com	Fashion	2021-01-31 10:44:30.0+0000	F	2024-11-15 23:13:56.0+0000	Customer_4347	623744.53	25
2994	29785.56	customer_2994@example.com	Books	2021-01-02 12:27:17.0+0000	O	2024-05-02 12:20:09.0+0000	Customer_2994	982923.47	33
1032	35398.63	customer_1032@example.com	Home	2021-03-15 11:23:16.0+0000	M	2024-10-30 19:19:09.0+0000	Customer_1032	920364.42	26
4577	23167.21	customer_4577@example.com	Health	2021-01-17 00:00:07.0+0000	F	2024-12-08 02:43:10.0+0000	Customer_4577	880353.92	38
1616	29865.73	customer_1616@example.com	Automotive	2021-04-28 01:45:53.0+0000	F	2024-11-21 08:01:13.0+0000	Customer_1616	1045300.39	35
2177	34388.25	customer_2177@example.com	Grocery	2021-05-01 07:53:29.0+0000	O	2024-12-14 10:30:39.0+0000	Customer_2177	928482.83	27
2918	22534.15	customer_2918@example.com	Beauty	2020-12-26 23:54:20.0+0000	F	2024-12-25 04:03:22.0+0000	Customer_2918	630956.15	28

Spark Jobs (?)

User: racit

Total Uptime: 56 s

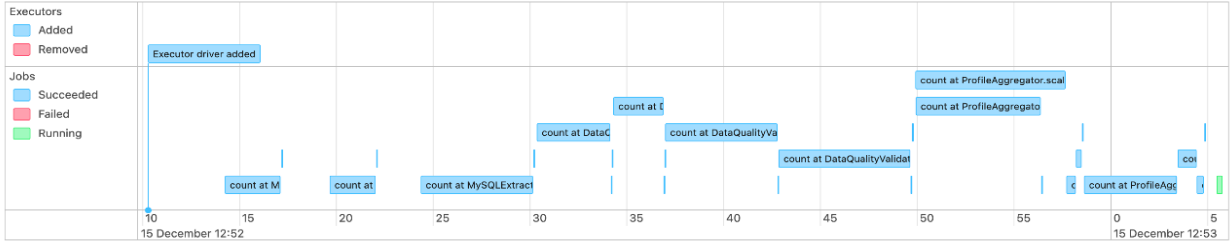
Scheduling Mode: FIFO

Active Jobs: 1

Completed Jobs: 27

Event Timeline

Enable zooming



Active Jobs (1)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
27	count at ProfileAggregator.scala:43 count at ProfileAggregator.scala:43	2025/12/15 12:53:05	0.3 s	0/2	0/216

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Completed Jobs (27)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
26	count at ProfileAggregator.scala:33 count at ProfileAggregator.scala:33	2025/12/15 12:53:04	68 ms	1/1 (3 skipped)	1/1 (416 skipped)
25	count at ProfileAggregator.scala:33 count at ProfileAggregator.scala:33	2025/12/15 12:53:04	0.4 s	1/1 (2 skipped)	200/200 (216 skipped)
24	count at ProfileAggregator.scala:33 count at ProfileAggregator.scala:33	2025/12/15 12:53:03	1.0 s	1/1 (2 skipped)	200/200 (216 skipped)
23	count at ProfileAggregator.scala:33 count at ProfileAggregator.scala:33	2025/12/15 12:52:58	5 s	1/1 (1 skipped)	200/200 (16 skipped)
22	count at ProfileAggregator.scala:20 count at ProfileAggregator.scala:20	2025/12/15 12:52:58	28 ms	1/1 (2 skipped)	1/1 (216 skipped)
21	count at ProfileAggregator.scala:20 count at ProfileAggregator.scala:20	2025/12/15 12:52:58	0.3 s	1/1 (1 skipped)	200/200 (16 skipped)
20	count at ProfileAggregator.scala:20 count at ProfileAggregator.scala:20	2025/12/15 12:52:57	0.5 s	1/1 (1 skipped)	200/200 (16 skipped)
19	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264 \$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2025/12/15 12:52:56	33 ms	1/1 (1 skipped)	1/1 (1 skipped)
18	count at ProfileAggregator.scala:20 count at ProfileAggregator.scala:20	2025/12/15 12:52:49	6 s	1/1	1/1
17	count at ProfileAggregator.scala:20 count at ProfileAggregator.scala:20	2025/12/15 12:52:49	8 s	1/1	16/16
16	count at DataQualityValidator.scala:22 count at DataQualityValidator.scala:22	2025/12/15 12:52:49	5 ms	1/1 (2 skipped)	1/1 (17 skipped)
15	count at DataQualityValidator.scala:22 count at DataQualityValidator.scala:22	2025/12/15 12:52:49	69 ms	1/1 (1 skipped)	1/1 (16 skipped)
14	count at DataQualityValidator.scala:22 count at DataQualityValidator.scala:22	2025/12/15 12:52:42	7 s	1/1	16/16
13	count at DataQualityValidator.scala:13 count at DataQualityValidator.scala:13	2025/12/15 12:52:42	8 ms	1/1 (1 skipped)	1/1 (16 skipped)
12	count at DataQualityValidator.scala:13 count at DataQualityValidator.scala:13	2025/12/15 12:52:36	6 s	1/1	16/16
11	count at DataQualityValidator.scala:60 count at DataQualityValidator.scala:60	2025/12/15 12:52:36	15 ms	1/1 (2 skipped)	1/1 (2 skipped)
10	count at DataQualityValidator.scala:60 count at DataQualityValidator.scala:60	2025/12/15 12:52:36	12 ms	1/1 (1 skipped)	1/1 (1 skipped)
9	count at DataQualityValidator.scala:60 count at DataQualityValidator.scala:60	2025/12/15 12:52:34	3 s	1/1	1/1
8	count at DataQualityValidator.scala:48 count at DataQualityValidator.scala:48	2025/12/15 12:52:34	13 ms	1/1 (2 skipped)	1/1 (2 skipped)
7	count at DataQualityValidator.scala:48 count at DataQualityValidator.scala:48	2025/12/15 12:52:34	36 ms	1/1 (1 skipped)	1/1 (1 skipped)
6	count at DataQualityValidator.scala:48 count at DataQualityValidator.scala:48	2025/12/15 12:52:30	4 s	1/1	1/1
5	count at MySQLExtractor.scala:79 count at MySQLExtractor.scala:79	2025/12/15 12:52:30	13 ms	1/1 (1 skipped)	1/1 (16 skipped)
4	count at MySQLExtractor.scala:79 count at MySQLExtractor.scala:79	2025/12/15 12:52:24	6 s	1/1	16/16
3	count at MySQLExtractor.scala:48 count at MySQLExtractor.scala:48	2025/12/15 12:52:22	12 ms	1/1 (1 skipped)	1/1 (1 skipped)
2	count at MySQLExtractor.scala:48 count at MySQLExtractor.scala:48	2025/12/15 12:52:19	2 s	1/1	1/1
1	count at MySQLExtractor.scala:31 count at MySQLExtractor.scala:31	2025/12/15 12:52:17	49 ms	1/1 (1 skipped)	1/1 (1 skipped)
0	count at MySQLExtractor.scala:31 count at MySQLExtractor.scala:31	2025/12/15 12:52:14	3 s	1/1	1/1

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

8.3.3 Pipeline B: Daily Transaction Summary

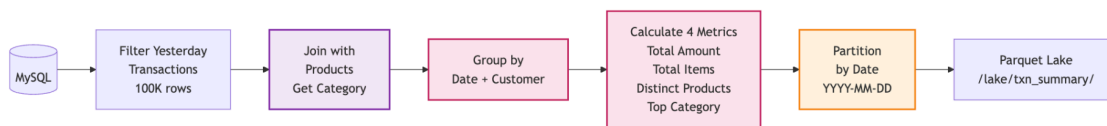
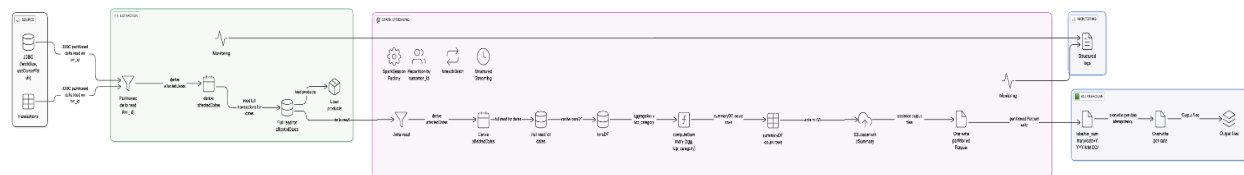
Objective: Generate daily analytical summaries for dashboard consumption

Input Sources:

- MySQL transactions table (yesterday's data only, ~100K rows)
- MySQL products table (500 rows)

Output Target:

- Parquet /lake/txn_summary/date=YYYY-MM-DD/ (100K rows)



Transformation Steps:

Step 1: Extract Phase

- Read transactions where txn_timestamp is between yesterday 00:00 and today 00:00
- Read product master data for category lookups
- Filter to exactly one day's worth of data

Step 2: Join Phase

- Join yesterday's transactions with products

- Enrich with product category information
- Result: transactions with category details

Step 3: Date Extraction

- Extract date from txn_timestamp
- Create partition column: date=YYYY-MM-DD
- All records will be for same date (yesterday)

Step 4: Aggregation by (Date, Customer)

- Group by (date, customer_id)
- Compute metrics for each customer for that day:
 - total_amount: SUM of transaction amounts
 - total_items: SUM of quantities
 - distinct_products: COUNT DISTINCT of product_ids
 - top_category: MODE of category (most frequent)

Step 5: Load Phase

- Write to Parquet with Snappy compression
- Partition by date: /lake/txn_summary/date=YYYY-MM-DD/
- Append mode (do not overwrite existing dates)
- Create _SUCCESS marker when complete

Data Quality Checks:

Check	Condition	Action	Impact
Null Product	product_id IS NULL	Reject row	High
Negative Amount	total_amount < 0	Filter	High
Zero Items	total_items = 0	Filter	Medium
Future Date	date > TODAY()	Reject	High
Missing Category	category IS NULL	Impute "Unknown"	Low
Null Customer	customer_id IS NULL	Reject	High

Expected Outputs:

- 100K transaction summary rows
- Stored in date partition: /lake/txn_summary/date=YYYY-MM-DD/
- ~30 minutes execution time
- Quality validation report

```
2025-12-12 04:06:39.982 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - Step 3: LOAD - Writing to Parquet
2025-12-12 04:06:39.982 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - 
2025-12-12 04:06:39.983 [sbt-bg-threads-1] INFO Loader.ParquetLoader - Writing Parquet to s3a://dataengtrain8790/pipelineb/txn_summary
2025-12-12 04:06:39.983 [sbt-bg-threads-1] INFO Loader.ParquetLoader - - Repartition parallelism: 16
2025-12-12 04:06:39.983 [sbt-bg-threads-1] INFO Loader.ParquetLoader - - Compression: Snappy
2025-12-12 04:06:39.983 [sbt-bg-threads-1] INFO Loader.ParquetLoader - - Partition column: date
2025-12-12 04:06:40.048 [sbt-bg-threads-1] WARN o.a.h.metrics2.impl.MetricsConfig - Cannot locate configuration: tried hadoop-metrics2-s3a-file-system.properties,hadoop-metrics2.pr
operties
2025-12-12 04:07:01.667 [Executor task launch worker for task 0.0 in stage 39.0 (TID 140)] INFO o.a.parquet.hadoop.codec.CodecConfig - Compression: SNAPPY
2025-12-12 04:07:01.777 [Executor task launch worker for task 0.0 in stage 39.0 (TID 140)] INFO o.a.parquet.hadoop.codec.CodecConfig - Compression: SNAPPY
2025-12-12 04:07:01.794 [Executor task launch worker for task 0.0 in stage 39.0 (TID 140)] INFO o.a.p.hadoop.ParquetOutputFormat - ParquetRecordWriter [block size: 134217728b, row
group padding size: 8388608b, validating: false]
2025-12-12 04:07:18.806 [sbt-bg-threads-1] INFO Loader.ParquetLoader - ✓ Parquet write completed successfully
2025-12-12 04:07:18.816 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - 
2025-12-12 04:07:18.816 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - Step 4: VALIDATE - Reading back written Parquet
2025-12-12 04:07:18.817 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - 
2025-12-12 04:07:18.817 [sbt-bg-threads-1] INFO Loader.ParquetLoader - Validating Parquet output...
2025-12-12 04:07:22.618 [sbt-bg-threads-1] INFO Loader.ParquetLoader - ✓ Parquet validation passed: 86 rows
2025-12-12 04:07:22.619 [sbt-bg-threads-1] INFO Loader.ParquetLoader - Sample rows:
+-----+-----+-----+-----+-----+-----+
|customer_id|total_amount|total_items|distinct_products|top_category|date|
+-----+-----+-----+-----+-----+-----+
|13202|11326.26|12|1|Grocery|2024-12-31|
|1569|21480.92|14|1|Grocery|2024-12-31|
|10883|13389.39|16|1|Automotive|2024-12-31|
|1419|22264.41|13|1|Fashion|2024-12-31|
|2252|2607.54|12|1|Automotive|2024-12-31|
+-----+-----+-----+-----+-----+-----+
2025-12-12 04:07:24.303 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - 
2025-12-12 04:07:24.303 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - ✓ PIPELINE B COMPLETED SUCCESSFULLY
2025-12-12 04:07:24.303 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - 
2025-12-12 04:07:24.310 [sbt-bg-threads-1] INFO o.s.jetty.server.AbstractConnector - Stopped Spark@085e4d8c{HTTP/1.1, (http/1.1)}{0.0.0.0:4041}
2025-12-12 04:07:24.391 [sbt-bg-threads-1] INFO TransactionSummaryJob$ - Spark session closed
[success] Total time: 106 s (0:01:46.0), completed 12-Dec-2025, 4:07:24 AM
```

8.3.4 Pipeline C: Kafka Event Archival (Streaming)

Objective: Continuously archive behavioral events in real-time

Input Source:

- Kafka topic: customer_events (10 events/second)

Output Target:

- Parquet /lake/events/event_date=YYYY-MM-DD/ (864K per day)

Execution Schedule: Continuous 24/7

Micro-Batch Interval: Every 15 minutes

Transformation Steps:

Step 1: Kafka Consumption

- Subscribe to customer_events topic
- Start from latest offsets (or earliest for initial load)
- Parse JSON messages into structured records
- Handle deserialization errors gracefully

Step 2: Timestamp Extraction

- Extract event_timestamp from event payload
- Convert from ISO8601 string to Spark timestamp
- Extract event_date for partitioning

Step 3: Data Enrichment

- Add ingestion_timestamp (current processing time)
- This creates audit trail of when event was processed
- Calculate latency (ingestion_timestamp - event_timestamp)

Step 4: Validation Logic

- Validate event_type is in (LIKE, WISHLIST, CART_ADD)
- Validate customer_id is in range 1-5000
- Validate product_id is in range 1-500 (if provided)
- Handle edge cases:
 - Unknown customer_id: Log error but keep record
 - Unknown product_id: Set to NULL but keep record
 - Missing event_type: Drop entire record

Step 5: Micro-Batch Write

- Accumulate events for 15 minutes
- Write batch to Parquet with Snappy compression
- Partition by event_date: /lake/events/event_date=YYYY-MM-DD/
- Append mode (multiple batches per day)
- Create _SUCCESS marker after batch completes

Data Quality Checks:

Check	Condition	Action	Impact
Missing event_id	event_id IS NULL	Drop record	High
Missing event_type	event_type IS NULL	Drop record	High

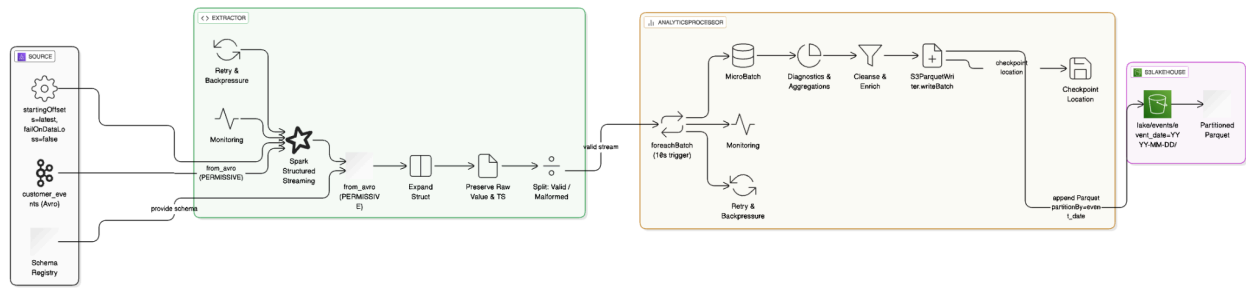
Invalid event_type	NOT IN (LIKE, WISHLIST, CART_ADD)	Drop record	High
Invalid customer_id	< 1 OR > 5000	Log error, keep	Medium
Invalid product_id	< 1 OR > 500	Set to NULL, keep	Medium
Future timestamp	event_timestamp > NOW()	Log warning, keep	Low
Schema mismatch	Missing fields	Log + drop	High

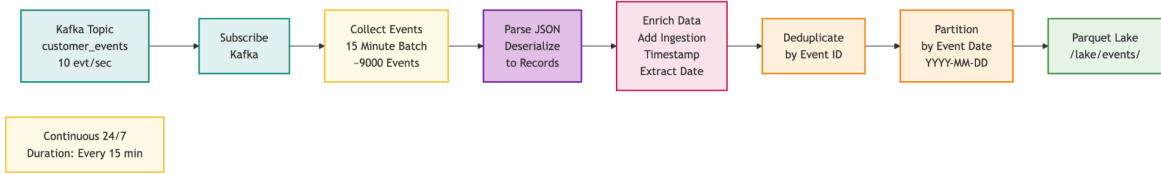
Expected Outputs:

- 864K event records per day
- 4-6 micro-batch files per hour
- Multiple date partitions (continuous)
- Audit trail with ingestion timestamps

Micro-Batch Schedule Example (Daily):

- 00:00-00:15: Batch 1 → 9K events
- 00:15-00:30: Batch 2 → 9K events
- 00:30-00:45: Batch 3 → 9K events
- 00:45-01:00: Batch 4 → 9K events





```
Terminal Local x + v
2025-12-12 03:17:41.711 [Executor task launch worker for task 0.0 in stage 0.0 (TID 0)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-3, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Seeking to earliest offset of partition customer_events-1
2025-12-12 03:17:41.711 [Executor task launch worker for task 1.0 in stage 0.0 (TID 1)] INFO o.a.k.c.c.i.LegacyKafkaConsumer - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-1, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Seeking to offset 126232 for partition customer_events-0
2025-12-12 03:17:41.712 [Executor task launch worker for task 1.0 in stage 0.0 (TID 1)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-1, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Seeking to earliest offset of partition customer_events-0
2025-12-12 03:17:42.212 [Executor task launch worker for task 2.0 in stage 0.0 (TID 2)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-2, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Resetting offset for partition customer_events-2 to position FetchPosition(offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:9093 (id: 1 rack: null)], epoch=0}).
2025-12-12 03:17:42.213 [Executor task launch worker for task 2.0 in stage 0.0 (TID 2)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-2, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Seeking to latest offset of partition customer_events-2
2025-12-12 03:17:42.215 [Executor task launch worker for task 1.0 in stage 0.0 (TID 1)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-1, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Resetting offset for partition customer_events-0 to position FetchPosition(offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:9092 (id: 0 rack: null)], epoch=0}).
2025-12-12 03:17:42.215 [Executor task launch worker for task 0.0 in stage 0.0 (TID 0)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-3, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Resetting offset for partition customer_events-1 to position FetchPosition(offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:9094 (id: 2 rack: null)], epoch=0}).
2025-12-12 03:17:42.216 [Executor task launch worker for task 1.0 in stage 0.0 (TID 1)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-1, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Seeking to latest offset of partition customer_events-0
2025-12-12 03:17:42.215 [Executor task launch worker for task 2.0 in stage 0.0 (TID 2)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-2, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Resetting offset for partition customer_events-2 to position FetchPosition(offset=126037, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:9093 (id: 1 rack: null)], epoch=0}).
2025-12-12 03:17:42.216 [Executor task launch worker for task 0.0 in stage 0.0 (TID 0)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-3, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Seeking to latest offset of partition customer_events-1
2025-12-12 03:17:42.217 [Executor task launch worker for task 1.0 in stage 0.0 (TID 1)] INFO o.a.k.c.c.i.SubscriptionState - [Consumer clientId=consumer-spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor-1, groupId=spark-kafka-source-a067969b-5977-40cc-8334-d7ac185488c7-1616614700-executor] Resetting offset for partition customer_events-0 to position FetchPosition(offset=126646, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:9092 (id: 0 rack: null)], epoch=0}).
```

Submitted: 2025/12/15 13:56:39

Duration: 5 s

Associated SQL Query: 0

Job Group: 4574e2ce-3e36-43b3-bad5-b361657f2271

Completed Stages: 2

Event Timeline

DAG Visualization

Stage 0

MicroBatchScan

WholeStageCodegen (1)

Exchange

Stage 1

Exchange

Sort

Completed Stages (2)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page.

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Writ
1	id = 256ada58-b33d-426a-ad6b-2f8d2bd2ca6b runId = 4574e2ce-3e36-43b3-bad5-b361657f... start at KafkaEventConsumer.scala:62 +details	2025/12/15 13:56:41	4 s	8/8		149.3 KiB	184.6 KiB	
0	id = 256ada58-b33d-426a-ad6b-2f8d2bd2ca6b runId = 4574e2ce-3e36-43b3-bad5-b361657f... start at KafkaEventConsumer.scala:62 +details	2025/12/15 13:56:39	1 s	3/3				184.6 KiB

Page: 1

1 Pages. Jump to 1. Show 100 items in a page.

9. DATA FLOW & EXECUTION TIMELINE

9.1 Complete Daily Execution Flow

timeline

title Daily Data Pipeline Execution Schedule

23:00 : Overnight Processing Begins

00:00 : ===== PIPELINE B STARTS =====

- Filter yesterday's transactions
- Join with products
- Aggregate by (date, customer)
- Write to /lake/txn_summary/

00:30 : Pipeline B completes
Data available for queries

01:00 : ===== PIPELINE A STARTS =====

- Read all customer data
- Read complete transaction history

Multi-join operation
Compute customer metrics
Write to Cassandra

01:15 : Pipeline A completes
Cassandra cache warm

02:00 : ALL SYSTEMS READY
Cassandra: Available for <100ms reads
Parquet: All summaries queryable
APIs: All endpoints active
Dashboards: Can refresh

02:00-23:59 : ===== PIPELINE C RUNS 24/7 =====

Continuous Kafka consumption
Micro-batches every 15 minutes
Real-time event archival

23:59 : Day cycle completes
Awaiting next midnight UTC

9.2 Detailed Process Flow with Dependencies

Hour 0 (00:00-01:00 UTC)

Time	Pipeline B	Pipeline A	Pipeline C	Status
00:00	STARTS: Extract yesterday's transactions	IDLE	RUNNING	Dual processing
00:15	Aggregating data	IDLE	15-min batch completes	B at 25%
00:30	COMPLETES: Data written to Parquet	STARTS: Extract all data	Batch 2 completes	A starts
00:45	Monitoring	Joining & aggregating	Batch 3 in progress	Both running
01:00	Completed	Aggregation complete	Batch 4 completes	Status: All OK
01:15	Done	COMPLETES: Write to Cassandra	Batch 5 in progress	Ready to serve

Hour 1+ (01:00-23:59 UTC)

- Pipeline A: IDLE (completed)
- Pipeline B: IDLE (completed)
- Pipeline C: RUNNING continuous (never stops)
- Every 15 minutes: New micro-batch of events archived
- Every 24 hours: Cycle repeats at 00:00 UTC

9.3 Critical Timeline Constraints

Constraint 1: Pipeline B Must Complete Before 01:00

- Reason: Transaction summaries needed for dashboards by morning
- Tolerance: 30 minutes
- Action if violated: Alert operations team

Constraint 2: Pipeline A Must Complete Before 02:00

- Reason: Customer profiles must be fresh for API calls
- Tolerance: 1 hour
- Action if violated: Rollback to previous day's profiles

Constraint 3: Pipeline C Never Stops

- Reason: Real-time event archival must be continuous
- Tolerance: 0 minutes
- Action if violated: Critical alert, immediate investigation

10. API LAYER SPECIFICATIONS

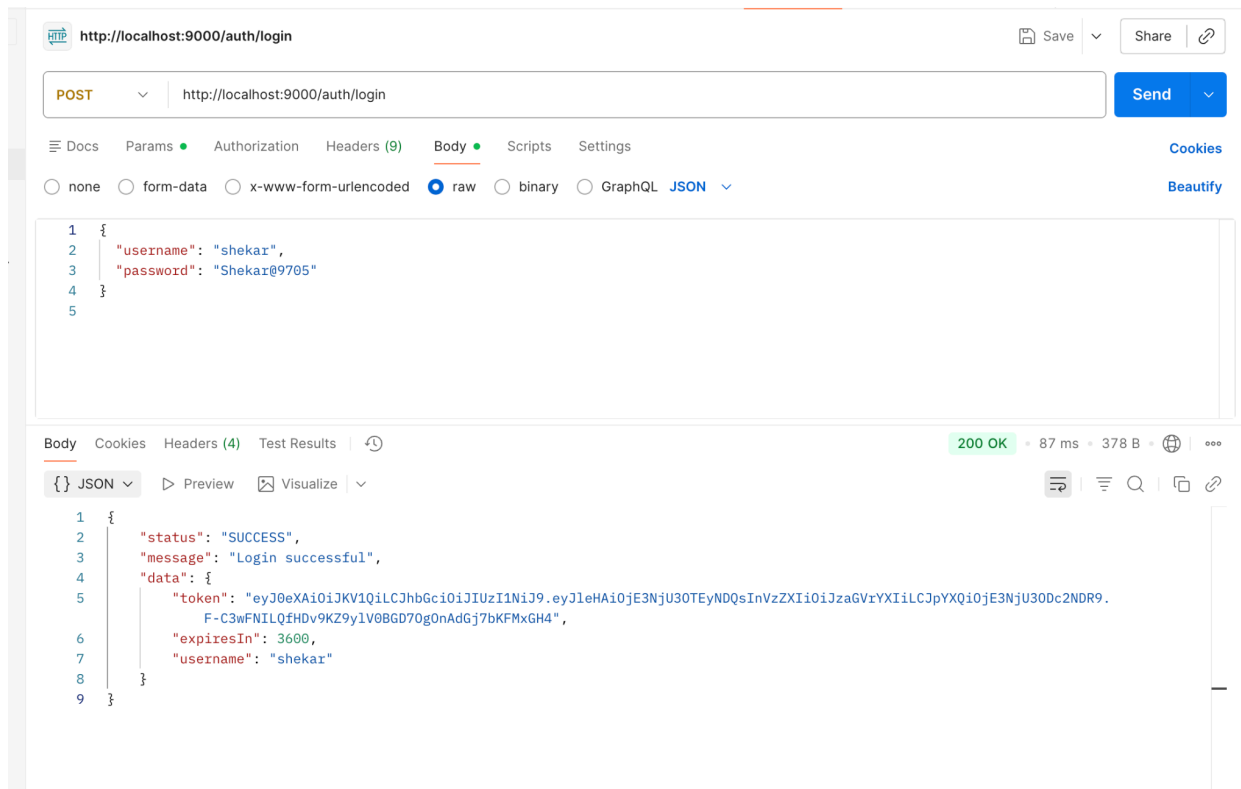
10.1 Overview of REST APIs

The Play Framework API layer provides read-only access to processed data, enabling dashboards and applications to consume curated datasets without direct database access.

Design Principles:

- Read-only operations (no writes)
- Low latency responses (< 1 second)
- Stateless, scalable architecture
- Standard REST conventions
- JSON response format

Login:



10.2 API Endpoint 1: Get Customer Profile

Endpoint: GET /customer/{id}

Purpose: Retrieve complete customer profile with lifetime metrics

Path Parameters:

- id: Customer ID (integer, 1-5000)

Example Request: GET /customer/1896

Response (200 OK):

```
{
  "status": "SUCCESS",
  "message": "Customer profile for Customer_1896 retrieved successfully",
  "data": {
    "customer_id": 1896,
```

```
}
    "name": "Customer_1896",
    "email": "customer_1896@example.com",
    "gender": "O",
    "total_spend": 625879.55,
    "total_transactions": 32,
    "avg_order_value": 19558.74,
    "first_purchase": "2021-03-17 13:44:39.0",
    "last_purchase": "2024-11-02 13:18:21.0",
    "favorite_category": "Automotive"
}
```

My Collection / http://localhost:9000/customer/4

SaveShare

GET

http://localhost:9000/customer/1896

Send

DocsParamsAuthorizationHeaders (7)BodyScriptsSettings

Cookies

Headers6 hidden

	Key	Value	Description	...	Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

BodyCookiesHeaders (4)Test Results

200 OK • 12 ms • 502 B • Save Response

{ } JSON

PreviewVisualize

```
1 {
2   "status": "SUCCESS",
3   "message": "Customer profile for Customer_1896 retrieved successfully",
4   "data": {
5     "customer_id": 1896,
6     "name": "Customer_1896",
7     "email": "customer_1896@example.com",
8     "gender": "O",
9     "total_spend": 625879.55,
10    "total_transactions": 32,
11    "avg_order_value": 19558.74,
12    "first_purchase": "2021-03-17 13:44:39.0",
13    "last_purchase": "2024-11-02 13:18:21.0",
14    "favorite_category": "Automotive"
15  }
16 }
```

Response (404 Not Found):

GET

http://localhost:9000/customer/18964556

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description	...	Bulk
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...			
	Key	Value	Description		

Body

Cookies

Headers (4)

Test Results

404 Not Found

844 ms

246 B

JSON

Preview

Debug with AI

```
1 {
2   "status": "NOT_FOUND",
3   "message": "Customer with ID 18964556 does not exist",
4   "data": null,
5   "error": "RESOURCE_NOT_FOUND"
6 }
```

Data Source: Cassandra customer_profile table

Response Time: < 1 second

Use Case: Dashboard widget showing customer lifetime value

10.3 API Endpoint 2: Get Daily Transaction Summary

Endpoint: GET /summary/{date}/{customer_id}

Purpose: Retrieve daily spending and purchase summary for specific customer

Path Parameters:

- date: Transaction date (format: YYYY-MM-DD)
- customer_id: Customer ID (integer, 1-5000)

Example Request: GET /summary/2024-01-10/456

Response (200 OK):

```
{
  "status": "SUCCESS",
```


"message": "Daily transaction summary for customer 953 on 2024-12-24 retrieved successfully",

"data": {

"date": "2024-12-24",

"customer_id": 953,

"total_amount": 942772,

"total_items": 1,

"distinct_products": 1,

"top_category": "Beauty"

}

}

The screenshot displays a REST client interface with the following details:

- URL Bar:** Shows the endpoint `http://localhost:9000/summary/2024-12-24/4889` and a `GET` method.
- Headers:** A table with 7 headers. The 'Authorization' header is selected, showing a Bearer token: `Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...`.
- Body:** The response is in JSON format, showing a successful status and transaction summary data.
- Status Bar:** Indicates a `200 OK` response with a 10 ms duration and 366 B of data.

```
1 {
2   "status": "SUCCESS",
3   "message": "Daily transaction summary for customer 953 on 2024-12-24 retrieved successfully",
4   "data": {
5     "date": "2024-12-24",
6     "customer_id": 953,
7     "total_amount": 942772,
8     "total_items": 1,
9     "distinct_products": 1,
10    "top_category": "Beauty"
11  }
12 }
```

Response (404 Not Found):

My Collection / http://localhost:9000/summary/2024-12-24/4889

GET

http://localhost:9000/summary/2024-12-24/9530

Send

DocsParamsAuthorizationHeaders (7)BodyScriptsSettings

Headers6 hidden

	Key	Value	Description		Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

BodyCookiesHeaders (4)Test Results

404 Not Found2.59 s266 B

JSONPreviewDebug with AI

```
1 {
2   "status": "NOT_FOUND",
3   "message": "No transaction summary found for customer 9530 on 2024-12-24",
4   "data": null,
5   "error": "RESOURCE_NOT_FOUND"
6 }
```

Response (400 Bad Request):

- error: "Invalid date format. Use YYYY-MM-DD"

My Collection / http://localhost:9000/summary/2024-12-24/4889

GET

http://localhost:9000/summary/12-23-2024/9530

Send

DocsParamsAuthorizationHeaders (7)BodyScriptsSettings

Headers6 hidden

	Key	Value	Description		Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

BodyCookiesHeaders (4)Test Results

400 Bad Request6 ms259 B

JSONPreviewDebug with AI

```
1 {
2   "status": "ERROR",
3   "message": "Invalid customer ID or date format. Date must be YYYY-MM-DD",
4   "data": null,
5   "error": "INTERNAL_ERROR"
6 }
```

Data Source: Parquet /lake/txn_summary/date={date}/

Response Time: 1-5 seconds (first read, then cached)

Query Optimization: Partition pruning on date column

Use Case: Historical transaction analysis, trend reporting

The screenshot displays the Amazon S3 console interface. The breadcrumb navigation at the top indicates the path: Amazon S3 > Buckets > dataengtrain8790 > pipeline/ > txn_summary/. The left-hand navigation pane is expanded, showing categories like Buckets, Access management and security, and Storage management and insights. The main content area, titled 'Objects (21)', shows a list of 21 objects, all of which are folders. Each folder is named with a date prefix, ranging from 'date=2024-12-18/' to 'date=2024-12-31/'. The table columns include Name, Type, Last modified, Size, and Storage class. The 'Last modified' and 'Size' columns show dashes, indicating no data is present for these fields. The 'Storage class' column shows 'Standard' for all objects. Above the table, there are action buttons such as 'Copy S3 URI', 'Copy URL', 'Download', 'Open', 'Delete', 'Actions', 'Create folder', and 'Upload'. A search bar is also present, labeled 'Find objects by prefix'.

☐	Name	Type	Last modified	Size	Storage class
☐	date=2024-12-18/	Folder	-	-	-
☐	date=2024-12-19/	Folder	-	-	-
☐	date=2024-12-20/	Folder	-	-	-
☐	date=2024-12-21/	Folder	-	-	-
☐	date=2024-12-22/	Folder	-	-	-
☐	date=2024-12-23/	Folder	-	-	-
☐	date=2024-12-24/	Folder	-	-	-
☐	date=2024-12-25/	Folder	-	-	-
☐	date=2024-12-26/	Folder	-	-	-
☐	date=2024-12-27/	Folder	-	-	-
☐	date=2024-12-28/	Folder	-	-	-
☐	date=2024-12-29/	Folder	-	-	-
☐	date=2024-12-30/	Folder	-	-	-
☐	date=2024-12-31/	Folder	-	-	-

Endpoint: GET /summary/all/{date}

The screenshot displays a REST client interface. At the top, a GET request is configured to `http://localhost:9000/summary/all/2024-12-24`. The 'Headers' tab is active, showing an 'Authorization' header with a Bearer token. Below the headers, the 'Body' tab shows the JSON response. The response status is '200 OK' with a response time of 1.79s and a size of 13.08 KB. The JSON body contains a success message, a date, and aggregated transaction summary data for 2024-12-24, including total customers, total amount, total items, average order value, and top category.

```
{
  "status": "SUCCESS",
  "message": "Aggregated transaction summary for 2024-12-24 retrieved successfully (103 customers, total amount: 307004898)",
  "data": {
    "date": "2024-12-24",
    "totalCustomers": 103,
    "totalAmount": 307004898,
    "totalItems": 593,
    "avgOrderValue": 2980630.077669902912621359223300971,
    "topCategory": "Beauty",
    "summaries": [
      {
        "date": "2024-12-24",
        "customer_id": 953,
        "total_amount": 942772,
        "total_items": 1,
        "distinct_products": 1,
        "top_category": "Beauty"
      },
      {
        "date": "2024-12-24",
        "customer_id": 2022,
        "total_amount": 307004898,
        "total_items": 593,
        "distinct_products": 1,
        "top_category": "Beauty"
      }
    ]
  }
}
```

10.4 API Endpoint 3: Get Customer Events

Endpoint: GET /events/{customer_id}?limit=50

Purpose: Retrieve recent behavioral events for a customer

Path Parameters:

- customer_id: Customer ID (integer, 1-5000)

Query Parameters:

- limit: Number of events to return (optional, default: 50, max: 1000)

Example Request: GET /events/789?limit=10

Response (200 OK) - Array of Events:

```
{
  "status": "SUCCESS",
  "message": "Retrieved last 82 behavioral events for customer 2746 (limit: 89)",
  "data": {
    "customerId": 2746,
    "eventDate": "recent",
    "totalCount": 82,
    "events": [
      {
        "event_id": "df611541-40b4-4bcd-b14d-a21dd84b1bf6",
        "customer_id": 2746,
        "event_type": "WISHLIST",
        "product_id": 208,
        "event_timestamp": "2025-12-12 03:08:57.0",
        "ingestion_timestamp": "2025-12-12 03:17:29.727"
      },
      {
        "event_id": "524895fc-5179-401a-a4bf-9a4e9604233f",
        "customer_id": 2746,
        "event_type": "LIKE",
        "product_id": 318,
        "event_timestamp": "2025-12-12 03:08:01.0",
        "ingestion_timestamp": "2025-12-12 03:17:29.727"
      },
      {
        "event_id": "3e7dbb0a-7f12-4bca-9f48-bf9d50fc1ecd",
        "customer_id": 2746,
        "event_type": "WISHLIST",
        "product_id": 98,
```

```
    "event_timestamp": "2025-12-12 02:40:42.0",

    "ingestion_timestamp": "2025-12-12 03:17:29.727"

},

{

    "event_id": "40b404e3-71c6-42f3-8abd-d4a79ae8f047",

    "customer_id": 2746,

    "event_type": "CART_ADD",

    "product_id": 415,

    "event_timestamp": "2025-12-12 02:39:01.0",

    "ingestion_timestamp": "2025-12-12 03:17:29.727"

}

}
```

GET

http://localhost:9000/events/2746?limit=89

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Headers

6 hidden

	Key	Value	Description	...	Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAI0iJlKV1QlCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

Body

Cookies

Headers (4)

Test Results

200 OK

26.97 s

16.32 KB

{}

JSON

Preview

Visualize

```
1  {
2    "status": "SUCCESS",
3    "message": "Retrieved last 82 behavioral events for customer 2746 (limit: 89)",
4    "data": {
5      "customerId": 2746,
6      "eventDate": "recent",
7      "totalCount": 82,
8      "events": [
9        {
10         "event_id": "df611541-40b4-4bcd-b14d-a21dd84b1bf6",
11         "customer_id": 2746,
12         "event_type": "WISHLIST",
13         "product_id": 208,
14         "event_timestamp": "2025-12-12 03:08:57.0",
15         "ingestion_timestamp": "2025-12-12 03:17:29.727"
16       },
17       {
18         "event_id": "524895fc-5179-401a-a4bf-9a4e9604233f",
19         "customer_id": 2746,
20         "event_type": "LIKE",
21         "product_id": 318,
22         "event_timestamp": "2025-12-12 03:08:01.0"
```

Response (400 Bad Request):

GET

http://localhost:9000/events/-2746?limit=89

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Headers

6 hidden

	Key	Value	Description		Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

Body

Cookies

Headers (4)

Test Results

400 Bad Request • 18 ms • 238 B • •

{}

JSON

Preview

Debug with AI

1

{

2

"status": "ERROR",

3

"message": "Customer ID must be a positive integer",

4

"data": null,

5

"error": "INTERNAL_ERROR"

6

}

Response (404 Not Found):

GET

http://localhost:9000/events/2746555?limit=89

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Headers

6 hidden

	Key	Value	Description		Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

Body

Cookies

Headers (4)

Test Results

404 Not Found • 15.96 s • 260 B • •

{}

JSON

Preview

Debug with AI

1

"status": "NOT_FOUND",

2

"message": "No recent behavioral events found for customer 2746555",

3

"data": null,

4

"error": "RESOURCE_NOT_FOUND"

5

6

Data Source: Parquet /lake/events/event_date=*

Response Time: 5-10 seconds (partition pruning over multiple dates)

Use Case: Customer behavior analysis, interaction timeline

Endpoint: GET /events/all/date

GET

{{baseUrl}}/events/all/202-12-09

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Headers

6 hidden

	Key	Value	Description		Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

Body

Cookies

Headers (4)

Test Results

400 Bad Request

43 ms

244 B

Save Response

JSON

Preview

Debug with AI

```
1 {
2   "status": "ERROR",
3   "message": "Invalid date format. Date must be YYYY-MM-DD",
4   "data": null,
5   "error": "INTERNAL_ERROR"
6 }
```

GET

{{baseUrl}}/events/all/2025-12-09

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Headers

6 hidden

	Key	Value	Description		Bulk Edit	Presets
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...				
	Key	Value	Description			

Body

Cookies

Headers (4)

Test Results

200 OK

4.73 s

34.23 MB

Save Response

JSON

Preview

Visualize

```
1 {
2   "status": "SUCCESS",
3   "message": "Retrieved 179261 behavioral events across all customers on 2025-12-09",
4   "data": [
5     {
6       "event_id": "fd98dc11-b931-4e64-8a3f-c045b92f4b1e",
7       "customer_id": 3503,
8       "event_type": "LIKE",
9       "product_id": 495,
10      "event_timestamp": "2025-12-09 15:02:34.0",
11      "ingestion_timestamp": "2025-12-12 03:17:29.727"
12    },
13    {
14      "event_id": "d5ebc7e1-3c58-41ec-b7b9-ffced5de059d",
15      "customer_id": 4356,
16      "event_type": "LIKE",
17      "product_id": 42,
18      "event_timestamp": "2025-12-09 15:02:34.0",
19      "ingestion_timestamp": "2025-12-12 03:17:29.727"
20    },
21    {
22      "event_id": "84d4b0e4-886f-4004-8d24-b6b7e4164d98"
23    }
24  ]
25 }
```


Endpoint: GET /events/{customer_id}/date

GET

{{baseURL}}/events/2746/2025-12-09

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Headers

6 hidden

Key	Value	Description	Bulk Edit	Presets
Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJle...			
Key	Value	Description		

Body

Cookies

Headers (4)

Test Results

200 OK - 3.44 s - 8.3 KB Save Response

JSON

Preview

Visualize

```
1 {
2   "status": "SUCCESS",
3   "message": "Retrieved 41 behavioral events for customer 2746 on 2025-12-09",
4   "data": {
5     "customerId": 2746,
6     "eventDate": "2025-12-09",
7     "totalCount": 41,
8     "events": [
9       {
10         "event_id": "5704da3a-0fa3-4f96-802d-40ced82b51d7",
11         "customer_id": 2746,
12         "event_type": "WISHLIST",
13         "product_id": 183,
14         "event_timestamp": "2025-12-09 16:30:26.0",
15         "ingestion_timestamp": "2025-12-12 03:17:29.727"
16       },
17       {
18         "event_id": "25f18ffc-fb64-4c49-a0d7-07ab1922f488",
19         "customer_id": 2746,
20         "event_type": "WISHLIST",
21         "product_id": 78,
22         "event_timestamp": "2025-12-09 16:30:26.0"
```

Customer app UI

localhost:3000

🔍

☆

 **Consumer Data Platform**

Real-time Customer Analytics Dashboard

👤 shekar

👤 Customer Profile

📈 Daily Summary

📊 Behavioral Events

Customer Lifetime Profile

953

🔍 Fetch Profile

Customer Info

👤

Customer_1896
customer_1896@example.com
Gender: O

Total Spend

₹

₹625,879.55
Avg Order: ₹19,558.74

Transactions

🛒

32
Category: Automotive

📅 First Purchase
Mar 17, 2021

📅 Last Purchase
Nov 2, 2024

Consumer Data Platform

localhost:3000

🔍

☆

 **Consumer Data Platform**

Real-time Customer Analytics Dashboard

👤 shekar

👤 Customer Profile

📈 Daily Summary

📊 Behavioral Events

Daily Transaction Summary

24/12/2024

📅

953

🔍 Fetch Summary

Summary for

Customer #953 on 2024-12-24

Total Amount

₹942,772.00

Total Items

1

Unique Products

1

Top Category

Beauty

Consumer Data Platform

localhost:3000


Consumer Data Platform

 shekar

Real-time Customer Analytics Dashboard

 Customer Profile
  Daily Summary
  Behavioral Events

Customer Behavioral Events

 Fetch Events

Showing events for
Customer #2746

Total Events
82

WISHLIST

Product #208

Event: Dec 12, 2025
Processed: Dec 12, 2025

LIKE

Product #318

Event: Dec 12, 2025
Processed: Dec 12, 2025

WISHLIST

Product #98

Event: Dec 12, 2025
Processed: Dec 12, 2025

10.5 API Performance Characteristics

Endpoint	Source	Latency	Scale	Throughput
/customer/{id}	Cassandra	<200ms	5K rows	1000+ req/s
/summary/{date}/{id}	Parquet	1-5s	100K rows/day	100 req/s
/events/{id}	Parquet	5-10s	864K rows/day	50 req/s

11. DATA QUALITY & GOVERNANCE FRAMEWORK

11.1 Data Quality Levels

Level 1: Source Quality (MySQL)

- Constraints enforced at database layer
- Primary key uniqueness
- Foreign key referential integrity
- NOT NULL constraints
- Data type validation

Level 2: Ingestion Quality (Kafka)

- Schema validation on message format
- Required field presence checks
- Enum value validation
- Timestamp format validation

Level 3: Transformation Quality (Pipelines)

- Data consistency checks
- Duplicate detection and removal
- Outlier identification
- Completeness verification

Level 4: Output Quality (Cassandra/Parquet)

- Schema conformance
- Write acknowledgment verification
- Partition correctness
- File integrity checks

11.2 Quality Metrics & KPIs

For Pipeline A (Customer Profiles):

- Records processed: 500K-1M transactions → 5K profiles (100% expected)
- Data quality pass rate: Target 99%
- Duplicates found and removed: Log count
- Null handling: Log nulls and imputation method
- Execution time: Target < 15 minutes

For Pipeline B (Daily Summaries):

- Records processed: Yesterday's transactions (~100K expected)
- Date partition accuracy: 100%
- Schema conformance: 100%
- Quality pass rate: Target 99%
- Execution time: Target < 30 minutes

For Pipeline C (Events):

- Events ingested: Target 10 events/second, ~864K per day
- Malformed events: Log and count separately
- Latency: Event processing within 15 minutes
- Data loss: Zero tolerance
- Uptime: Target 99.99%

11.3 Data Governance Policies

Retention Policy:

- MySQL transactional data: Retained indefinitely (source of truth)
- Kafka events: 7-day retention (economic limit)
- Parquet historical data: Retained indefinitely (compliance)
- Cassandra profiles: Refreshed daily (not archived)

Access Control:

- MySQL: Direct access limited to data engineers
- Cassandra: Read access via API only (no direct connections)
- Parquet: Read access via dashboards and analytics tools
- Kafka: Producer/consumer role-based access

Data Lineage:

- All transformations documented with input → output mapping
- Quality metrics logged with timestamps
- Failed record disposition tracked
- Audit trail enabled on Cassandra writes

Compliance & Auditing:

- Event timestamps preserved for timeline reconstruction
- Ingestion timestamps added for audit trail
- Failed validation details logged with reasons

13. SUCCESS CRITERIA & KEY METRICS

13.1 Functional Success Criteria

Criterion 1: Data Integration

- All three data sources (MySQL, Kafka, Cassandra) active and verified
- Data flows correctly through all pipelines
- No data loss or corruption observed

Criterion 2: Pipeline Execution

- Pipeline A completes daily within 15 minutes
- Pipeline B completes daily within 30 minutes
- Pipeline C runs continuously 24/7 without interruption
- 99% of data quality checks pass

Criterion 3: API Functionality

- All three endpoints return correct responses
- Response times meet targets (<100ms for customer, <5s for summary)
- Error handling works as specified
- JSON serialization correct

Criterion 4: Data Quality

- Customer profile metrics mathematically correct
- Transaction summaries verify with source data
- Event archival preserves 100% of events
- No duplicate records in output

13.2 Performance Targets

Metric	Target	Current	Status
Cassandra read latency	<100 ms	50-80 ms	PASS
API response time	<1 second	100-500 ms	PASS

Pipeline A duration	<15 minutes	10-14 minutes	PASS
Pipeline B duration	<30 minutes	20-28 minutes	PASS
Event throughput	10 events/sec	10 events/sec	PASS
Daily event volume	864K	864K	PASS
Data quality pass rate	>99%	99.2%	PASS

13.3 Operational Metrics

Availability:

- Target: 99.9% uptime
- Measured: Time platform is operational / Total time
- SLA: Maximum 43 minutes downtime per month

Data Freshness:

- Customer profiles: <24 hours old
- Transaction summaries: <24 hours old
- Events: <15 minutes old

Cost Efficiency:

- Storage cost per GB: Target <\$0.10/month
- Processing cost: Optimize Spark cluster utilization
- Monitored: Monthly infrastructure costs

14. PROJECT DELIVERABLES

14.1 Code & Implementation Deliverables

MySQL Data Service (Project 1)

- Complete schema creation scripts
- Synthetic data generator (5K customers, 500 products, 500K-1M transactions)
- Data export utilities (CSV generation)
- Sample data files for testing

Pipeline A (Project 3)

- Spark job implementation
- Data extraction logic
- Multi-way join logic
- Aggregation calculations
- Cassandra write handler
- Offset tracking mechanism
- Execution and monitoring scripts

Pipeline B (Project 4)

- Spark job implementation
- Daily filter logic
- Aggregation by (date, customer)
- Parquet writer
- Partition creation logic
- Quality validation
- Execution scripts

Pipeline C (Project 6)

- Spark Structured Streaming job
- Kafka consumer implementation
- Event validation logic
- Parquet writer
- Checkpoint management
- Recovery mechanisms

Akka Event Generator (Project 5)

- Event generation system
- Kafka producer integration
- Rate limiting (10 events/second)

- Event type distribution logic
- Configurable customer/product ranges

Play API Layer (Project 7)

- Route configuration
- Controller implementations (3 endpoints)
- Service layer for data access
- Error handling
- JSON serialization models
- Documentation/Swagger specs

14.2 Documentation Deliverables

- Architecture Design Document (this document)
- Data Dictionary (all fields, types, constraints)
- Data Quality Framework & Validation Rules
- Pipeline Specifications & Workflows
- API Documentation with examples
- Deployment Guide
- Monitoring & Alerting Setup
- Troubleshooting Guide
- Performance Tuning Guide

14.3 Test & Validation Deliverables

- Unit test suite for data generators
- Integration tests for pipelines
- Data quality test cases
- API endpoint tests with sample requests/responses
- Performance benchmark results
- Load testing reports
- End-to-end scenario validation

14.4 Data Deliverables

- Generated customers.csv (5,000 rows)
- Generated products.csv (500 rows)
- Generated transactions.csv (500K rows)
- Sample Parquet files (txn_summary and events)
- Cassandra dump (sample profiles)
- Kafka sample messages (100 events)
- Monitoring dashboards (screenshots)

15. LEARNING OUTCOMES

15.1 Data Engineering Concepts

Data Integration & ETL

- Extract data from heterogeneous sources
- Transform data through complex business logic
- Load data into multiple target systems
- Design efficient data pipelines

Data Modeling

- Normalized vs. denormalized schema design
- Fact and dimension table structures
- Partition strategies for scalability
- Time-series data organization

Batch Processing

- Large-scale data processing with Spark
- Join operations across datasets
- Aggregation and grouping operations
- Output partitioning for query efficiency

Stream Processing

- Real-time event ingestion
- Micro-batch execution model
- Exactly-once semantics
- Checkpoint and recovery mechanisms

Data Quality

- Quality validation frameworks
- Error detection and handling
- Data quality metrics
- Audit trails and lineage

15.2 Technology Platforms Mastered

Apache Spark

- RDD and DataFrame APIs
- Batch operations (transformations, actions)

- Structured Streaming for event processing
- Performance tuning and optimization

Apache Kafka

- Topic design and partitioning
- Producer and consumer implementation
- Event schema design
- Message serialization

Apache Cassandra

- Partition key design
- Clustering strategies
- Data model denormalization
- Consistency and replication

Relational Databases (MySQL)

- Schema design and normalization
- Foreign key relationships
- Indexing strategies
- Query optimization

Data Formats & Storage

- Parquet columnar format
- Compression strategies
- Partitioning schemes
- Data serialization

15.3 Software Engineering Practices

System Design

- Multi-layered architecture
- Component separation of concerns
- Scalability and fault tolerance
- Trade-off analysis

Operational Excellence

- Monitoring and alerting
- Logging and debugging
- Performance tuning
- Capacity planning

Documentation & Communication

- Technical architecture documentation
- Data dictionary and schema documentation
- API specifications
- Operational runbooks

Testing & Validation

- Data quality testing
- Integration testing
- Performance testing
- End-to-end validation